Sprint 16 · Reto «Entrega final: evaluar, demostrar y documentar»

Esta es la ficha completa del reto, explicada paso a paso para quien nunca ha trabajado con esta metodología ni con IA. Léela entera **antes de empezar** y vuelve a ella cuando tengas dudas.

- Si es tu primer reto, lee antes Cómo se trabaja un reto.
- Volver al README del sprint · Guía del sprint en PDF · versión ecofriendly para imprimir.
- **Este reto en PDF:** a color · ecofriendly, para imprimir.

RETO Ficha del reto

En pocas palabras (léelo primero)

Qué os pide el cliente. El cliente (el docente) va a **recibir vuestro trabajo como si fuera un entregable real**. Quiere poder:

- **clonar el repositorio** y seguir el README,
- **ver funcionar el demostrador**,
- **leer una memoria de 10–15 páginas**,
- y **entender los riesgos** antes de decidir si adopta vuestro modelo.

Este es el sprint de la entrega final. Todo lo que haya en la rama principal del repositorio **a las 14:30 del 10/12** es lo que se evaluará. Pasada esa hora, **se congela**.

Qué vais a hacer, en seis pasos:

1. **Evaluar a fondo** el modelo candidato (una sola vez en el test) con visualizaciones y análisis de errores.
2. **Analizar sesgos** por subgrupos y **aplicar y medir una medida de mitigación**.
3. **Cerrar el registro de riesgos** y clasificar el sistema según RGPD y AI Act.
4. **Construir un demostrador ejecutable** que no se rompa con entradas erróneas, avise de la incertidumbre y tenga **plan B en vídeo**.
5. **Documentar:** model card, README reproducible y **memoria técnica de 10–15 páginas** (esqueleto).
6. **Congelar** (etiqueta `v1.0-entrega`; lista de congelación) y entregar la **coevaluación individual** (formulario).

Qué vais a aprender haciéndolo: a **cerrar un proyecto de IA de forma profesional**: evaluar con honestidad, medir sesgos, construir una demo robusta y documentar para que otra persona lo reproduzca.

Cuánto tiempo: 18 horas, en tres sesiones de clase (**4, 9 y 10/12**, de 6 horas). **No hay clase el 7 y el 8/12.**

Palabras que van a aparecer en este reto

Palabra	Qué significa, en sencillo
Congelación	Momento a partir del cual no se puede cambiar nada. Se evalúa lo que haya
Etiqueta (tag) `v1.0-entrega`	Marca en Git que señala la versión final entregada
Análisis de errores	Estudiar en qué casos falla el modelo y por qué
Subgrupo	Una parte de los datos (por canal, país, tipo de imagen…)
Sesgo	Que el modelo funcione peor para un grupo que para otros
Mitigación	Medida para reducir un sesgo o un riesgo, que luego hay que **medir**
Incertidumbre	Cuánto se puede equivocar el modelo y cómo se comunica
Demostrador	Una app o cuaderno interactivo donde alguien puede probar el modelo
Streamlit / Gradio	Herramientas para crear una app web sencilla desde Python
Plan B	Una alternativa si la demo falla (un vídeo grabado)
Model card	Ficha técnica del modelo: usos, datos, métricas, límites, riesgos
Memoria técnica	Documento de 10–15 páginas que cuenta el proyecto completo
Reproducible	Otra persona puede ejecutar todo desde cero y obtener lo mismo

Palabra	Qué significa, en sencillo
Coevaluación	Valoración individual del trabajo de cada compañero y compañera
`pytest`	Herramienta que ejecuta las pruebas automáticas del código

El cliente

El cliente (el docente) va a **recibir vuestro trabajo como si fuera un entregable real**: quiere clonar el repositorio, seguir el README, ver funcionar el demostrador, leer una memoria de 10-15 páginas y entender los riesgos antes de decidir si lo adopta.

Qué se entrega (todo congelado el 10/12 a las 14:30)

1. **Evaluación final** del modelo candidato en el test, con **visualizaciones** y **análisis de errores** (`notebooks/04_evaluacion.ipynb`).
2. **Análisis de sesgos y limitaciones** por subgrupos, con **medida de mitigación aplicada y evaluada**.
3. **Registro de riesgos éticos cerrado** y clasificación RGPD / AI Act.
4. **Demostrador ejecutable** (Streamlit / Gradio / notebook interactivo / endpoint), con manejo de errores del usuario, aviso de incertidumbre y **plan B en vídeo**.
5. **Repositorio reproducible:** README, `requirements.txt` con versiones fijas, datos o script de descarga, pruebas, commits de todos.
6. **Model card** y **memoria técnica** (10-15 páginas, PDF).
7. **Coevaluación** individual (anexo B) rellenada por cada persona.

Restricciones

- **A las 14:30 del 10/12 se congela el repositorio y la memoria.** Se evaluará lo que haya en la rama principal en ese momento (etiqueta `v1.0-entrega`).
- Memoria de **10 a 15 páginas** (los anexos no cuentan). Autocontenida y propia (no copiada).
- El demostrador debe funcionar **desde cero** en otro equipo.
- Sin datos personales, claves ni credenciales en el repositorio.

Criterios de evaluación

Rúbrica del proyecto (100 puntos). En este sprint pesan:

Criterio	Qué se mira
D · Evaluación y visualización (10)	Análisis de errores, gráficos claros, interpretación del modelo, incertidumbre
E · IA responsable (10)	Sesgos, limitaciones, riesgo RGPD / AI Act, mitigación aplicada **y evaluada**
F · Demostrador y reproducibilidad (10)	Funciona, se reproduce desde el README, versiones fijas, pruebas, errores del usuario
G · Equipo (10)	Sprints completos, roles rotados, commits repartidos, mejora entre retros, coevaluación
H · Memoria (5)	Completa, clara, con figuras, referencias y conclusiones accionables
C · Modelado (20)	Rigor de la comparativa y de la justificación (se cierra aquí)

La coevaluación y el historial de commits ajustan la nota individual con un factor de 0,8 a 1,1.

Pasos sugeridos

Cuándo	Paso
4/12	Evaluación final en test (una sola vez). Gráficos de la plantilla `S13_02` adaptados. Métricas por subgrupos. Lista de errores más frecuentes con ejemplos.
9/12	Mitigación de sesgos + medición del efecto. Demostrador funcionando con la plantilla de `S16_01`. Model card. Borrador de la memoria (secciones 1-6).
10/12	Memoria completa; **prueba de reproducibilidad en un ordenador limpio**; vídeo plan B; checklist de congelación; coevaluación; retro.

Guía paso a paso para principiantes

Paso 1 · Evaluación final y errores (4/12)

1. **Evaluad en el test, una sola vez**, el modelo candidato del Sprint 15. Anotad las métricas con su incertidumbre.

2. Usad los **gráficos de la plantilla `S13_02`** (adaptadlos): matriz de confusión, curvas, importancia de variables, SHAP o Grad-CAM.
3. **Análisis de errores:** sacad una lista de los casos donde falla más y mirad ejemplos reales. ¿Qué tienen en común?
4. **Métricas por subgrupos:** calculad el rendimiento para cada subgrupo relevante (por canal, país, tipo de imagen…). Si hay diferencias grandes, hay un sesgo.
5. Guardad todo en `notebooks/04_evaluacion.ipynb`.

Paso 2 · Mitigación de sesgos y demostrador (9/12)

1. **Elegid una medida de mitigación** de un sesgo detectado (por ejemplo, reponderar datos, ajustar el umbral por grupo, añadir datos del grupo infrarrepresentado, o limitar el uso del modelo). **Aplicadla y medid su efecto**: ¿mejora el subgrupo? ¿empeora algo?
2. **Demostrador:** partid de la plantilla del cuaderno `S16_01` (carpeta `plantillas/demostrador/`: `logica.py`, `app.py`, `api.py`, `requirements.txt`, `README_demostrador.md`). Debe: - **validar las entradas del usuario** (si pone un dato imposible, avisar con un mensaje claro), - **avisar de la incertidumbre** («confianza: media»), - funcionar **desde cero en otro ordenador**.
3. **Pruebas automáticas** con `pytest -q` para la lógica.
4. **Model card** con la plantilla `plantilla_model_card.md`.
5. **Borrador de la memoria** (secciones 1–6).

Paso 3 · Cierre y congelación (10/12, antes de las 14:30)

1. **Memoria completa** (10–15 páginas, sin contar anexos) en PDF dentro de `docs/`. Usad `plantilla_esqueleto_memoria_tecnica.md`: tiene un presupuesto de páginas y un mapa de los requisitos R1–R11.
2. **Prueba de reproducibilidad en un ordenador limpio** (o un Colab nuevo): otra persona del equipo sigue el README y comprueba que funciona. **Corregid lo que falle.**
3. **Vídeo plan B** del demostrador (1–2 minutos).
4. **Cerrad el registro de riesgos** y la clasificación RGPD / AI Act.
5. Pasad el `checklist_congelacion.md`.
6. **Cread la etiqueta `v1.0-entrega`** en Git **antes de las 14:30**.
7. **Cada integrante entrega su coevaluación individual** (anexo B, `plantilla_coevaluacion.md`).
8. Review (4 minutos de demostrador) y retro.

Cómo usar la IA en este reto

Para qué	Ejemplo de petición	Qué tenéis que comprobar
Entender sesgos	«Explícame con un ejemplo qué significa que un modelo tenga peor rendimiento en un subgrupo y qué medidas existen»	Que podáis aplicar una **y medirla**
Crear la app	«Ayúdame a añadir validación de entradas a esta app de Streamlit: *[código]*»	Que **probáis** entradas absurdas
Estructurar la memoria	«Dame un guion de las secciones de una memoria técnica de un proyecto de IA»	Que cada sección **la redactáis vosotros** con **vuestros resultados**
Revisar la redacción	«Corrige la ortografía y la claridad de este párrafo sin cambiar el contenido»	Que no se haya cambiado ninguna cifra

Reglas del reto:

- **A las 14:30 del 10/12 se congela** el repositorio y la memoria. Se evalúa la rama principal en ese momento (etiqueta `v1.0-entrega`).
- La memoria debe ser **autocontenida y propia** (no copiada). **Si usáis IA para redactar, revisadla y declaradlo** en la memoria.
- **No debe haber datos personales, claves ni credenciales** en el repositorio.
- El demostrador debe funcionar **desde cero** en otro equipo.
- La **coevaluación y los commits** ajustan la nota individual (factor entre 0,8 y 1,1).

Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Dejar la memoria para el último día	«Total, ya está hecho»	Borrador el 9/12 y revisión el 10/12
El demostrador solo funciona en el ordenador de una persona	Dependencias sin documentar	`requirements.txt` con versiones y prueba en ordenador limpio

Error	Por qué ocurre	Cómo evitarlo
Mitigación que se aplica pero no se mide	Se piensa que basta con hacerla	Comparad métricas **antes y después**, por subgrupo
Subir cambios después de las 14:30	Se siguen retocando cosas	Crear la etiqueta antes y no tocar
Coevaluación sin entregar	Se olvida	Cada persona la entrega; cuenta para la nota individual
Memoria con más de 15 páginas o con fragmentos copiados	Se llena de relleno	Respetad el presupuesto de páginas; los anexos no cuentan

Un buen resultado se parece a...

- **Análisis de sesgos:** *«El recall baja del 74 % global al 58 % en reservas de canal móvil. Aplicamos reponderación y subió al 67 % sin perjudicar al resto (−1 punto)»* (cifras de ejemplo).
- **Demostrador:** al escribir una edad de 250, muestra *«Introduce una edad entre 0 y 120»*; al predecir, muestra *«Riesgo: alto (confianza media)»*.
- **Repositorio:** alguien externo lo clona, ejecuta tres comandos y ve la demo funcionando.

Antes de entregar: autoevaluación rápida

Pregunta	Sí / No
¿La evaluación en test se hizo una sola vez, con gráficos y análisis de errores?	
¿Hay rendimiento por subgrupos y una mitigación aplicada **y medida**?	
¿El registro de riesgos está cerrado y hay clasificación RGPD / AI Act justificada?	
¿El demostrador funciona desde cero, gestiona entradas erróneas y avisa de la incertidumbre?	
¿Hay pruebas (`pytest -q`) y vídeo plan B?	
¿`requirements.txt` con versiones y README probado por otra persona del equipo?	
¿Model card y memoria (10–15 páginas) en PDF dentro de `docs/`?	
¿Etiqueta `v1.0-entrega` creada antes de las 14:30?	
¿Cada integrante ha entregado su coevaluación?	

Si os atascáis

- **El demostrador no arranca en otro ordenador:** faltan dependencias. Generad `requirements.txt` con `pip freeze` (o a mano) y probad en un entorno nuevo.
- **No sabemos qué sesgo medir:** mirad las columnas con categorías (país, canal, tipo) y calculad la métrica para cada valor.
- **Falta tiempo:** priorizad lo obligatorio (R1–R11 y checklist de congelación). Es mejor una memoria honesta con límites que una incompleta.
- Más ayuda: Cómo se trabaja un reto.

Qué se enseña en la review

Demostración de 4 minutos del demostrador (con una entrada errónea a propósito), una diapositiva con el análisis de sesgos y la medida aplicada, y la lista de riesgos altos con su estado. El docente propone **una entrada difícil** para probar el demostrador en directo.

Definition of Done (lista de comprobación)

Si queda una casilla sin marcar, **el reto no está terminado**.

- ☐ Evaluación final en test hecha **una sola vez**, con gráficos y análisis de errores.
- ☐ Rendimiento **por subgrupos** y medida de mitigación aplicada y medida.
- ☐ Registro de riesgos cerrado; clasificación RGPD / AI Act justificada.
- ☐ Demostrador funcionando desde cero, con entradas erróneas gestionadas y aviso de incertidumbre.
- ☐ Pruebas (`pytest -q`) y vídeo plan B.
- ☐ `requirements.txt` con versiones; README reproducible probado por otra persona del equipo.
- ☐ Model card y memoria (10-15 páginas) en PDF dentro de `docs/`.

- [ ] Etiqueta `v1.0-entrega` creada antes de las 14:30 del 10/12.
- [ ] Coevaluación entregada por **cada** integrante; retro hecha.